# Multi-class Classification and Model Selection

A network that predicts which of four tiers a customer belongs to, chosen among 18 configurations on validation rows and measured once on a separate test file.


01 Multi-class classification
02 Per-class metrics
03 Feature scaling
04 Optimizers
05 Regularisation
06 Model selection


**Scope.** Tensors, autograd, `nn.Module`, `Dataset` and `DataLoader`, a binary classifier trained on minibatches, a validation split, early stopping and the encoding of a table as a feature matrix with every statistic taken from the training rows are assumed.

After reading it you should be able to do the following without looking anything up.

- Write out the softmax and the cross-entropy and compute `CrossEntropyLoss` for one row by hand.
- Compute the precision, the recall and the F1 of every class from a confusion matrix.
- Explain from the loss and the weights what unscaled features do to gradient descent.
- Write out the updates of SGD, SGD with momentum and Adam and compute their first steps by hand.
- Write out weight decay and dropout and read their effect on the training and validation loss.
- Choose a network on validation rows and measure it once on a separate test file.

Every example assumes `import copy`, `import csv`, `import itertools`, `import numpy as np`, `import torch`, `import torch.nn as nn` and `from torch.utils.data import DataLoader, TensorDataset`. Values in `# ->` comments are rounded to the digits shown.

01

## Multi-class classification

We turn the customer rows into features and predict which of four tiers each customer belongs to. A network for four classes outputs one score per class, called a logit. The label of a row is the index of its class, an integer. The softmax turns the logits of a row into probabilities. The loss of a row is minus the logarithm of the probability given to its true class. We leave the softmax out of the network because `CrossEntropyLoss` applies it to the logits itself.

### THE TWO FILES

```
def read_rows(path):
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))        # one dictionary per row, every value a string

train = read_rows("train.csv")      # the rows the network is fitted and validated on
test = read_rows("test.csv")        # held out, to measure the chosen network once
len(train), len(test)               # -> (3660, 1000)
train[0]
# -> {'age': '33', 'income': '23750.15', 'tenure_months': '32', 'score': '26.4',
#     'region': 'Midland', 'plan': 'legacy', 'channel': 'shop', 'churn': '0',
#     'tier': 'bronze'}
```

`train.csv` holds 3660 customers. `test.csv` holds 1000 further customers. Both files have the columns of `train[0]`. The label `tier` is `bronze`, `silver`, `gold` or `platinum`. The column `churn` is 1 for a customer who

left and 0 for one who stayed.

```
NUMERIC = ["age", "income", "tenure_months", "score"]
CATEGORIES = {"region": ["North", "Harbour", "Midland", "Eastfield", "Southgate", "Upland"],
              "plan": ["basic", "standard", "premium", "legacy"],
              "channel": ["shop", "phone", "online"]}

def column(rows, name):
    return np.array([float(r[name]) if r[name] != "" else np.nan for r in rows])

def features(rows, fit):
    medians = {c: np.nanmedian(column(fit, c)) for c in NUMERIC}

    def filled(rs):                                          # gaps filled with the medians
        return np.stack([np.where(np.isnan(column(rs, c)), medians[c], column(rs, c))
                         for c in NUMERIC], axis=1)

    mean, std = filled(fit).mean(axis=0), filled(fit).std(axis=0)
    blocks = [(filled(rows) - mean) / std]                   # standardised
    for cat, names in CATEGORIES.items():                    # one 0/1 column per category
        v = np.array([r[cat] for r in rows])
        blocks.append(np.stack([(v == n).astype(float) for n in names], axis=1))
    return np.concatenate(blocks, axis=1)                    # 4 + 6 + 4 + 3 = 17 columns
```

`features(rows, fit)` fills the gaps of the four numeric columns of `rows` with the medians of `fit`. It standardises the filled columns with the means and standard deviations of `fit`. It adds one 0/1 column per category, in the order of `CATEGORIES`.

#### A VALIDATION SPLIT AND INTEGER LABELS

```
TIERS = ["bronze", "silver", "gold", "platinum"]

rng = np.random.default_rng(42)
idx = rng.permutation(len(train))                # 0 to 3659 in a random order
val_rows = [train[i] for i in idx[:732]]         # a random fifth: validation
fit_rows = [train[i] for i in idx[732:]]         # the rest: fitting

def tensors(rows):
    x = torch.tensor(features(rows, fit_rows), dtype=torch.float32)     # (n, 17)
    y = torch.tensor([TIERS.index(r["tier"]) for r in rows])            # (n,) int64
    return x, y

(xf, yf), (xv, yv), (xt, yt) = tensors(fit_rows), tensors(val_rows), tensors(test)
len(yf), len(yv), len(yt)       # -> (2928, 732, 1000)
torch.bincount(yf)              # -> tensor([726, 750, 731, 721])   fitting rows per tier
[sum(r["tier"] == t for r in train) for t in TIERS]     # -> [915, 915, 915, 915]
[sum(r["tier"] == t for r in test) for t in TIERS]      # -> [250, 250, 250, 250]
```

Both files hold the four tiers equally often, 915 and 250 times. A random fifth of `train.csv`, 732 rows, is held back for validation. The network is fitted on the other 2928 rows. Choices such as the number of epochs are made on the validation rows. The 1000 rows of `test.csv` measure the network finally chosen, once. All three sets are encoded with the statistics of the fitting rows. The label of a row is the position of its tier in `TIERS`, an integer from 0 to 3. `torch.tensor` turns a list of Python integers into an int64 tensor. PyTorch also calls this dtype `long`.

```
class TierNet(nn.Module):
    def __init__(self, n_in):
        super().__init__()
        self.stack = nn.Sequential(
            nn.Linear(n_in, 48), nn.ReLU(),
            nn.Linear(48, 24), nn.ReLU(),
            nn.Linear(24, 4))               # FOUR logits, no softmax here

    def forward(self, x):
        return self.stack(x)                # (n, 17) -> (n, 4), no squeeze

loss_fn = nn.CrossEntropyLoss()             # logits and integer labels
```

The last layer has one unit per class. The output holds one row of four logits for every input row. It is not squeezed. The label is a `long` holding the class index. It is neither a float nor a one-hot row. A float label raises an error:

```
logits = torch.tensor([[2.0, 1.0, 0.1, 0.0]])       # (1, 4): one row, four logits
nn.CrossEntropyLoss()(logits, torch.tensor([0.0]))
# RuntimeError: expected target dtype to be Long or Byte, but got Float
```

#### SOFTMAX, CROSS-ENTROPY AND NLLLOSS

For the logits $z_0$ to $z_{C-1}$ of one row, the softmax gives the probability of class k as

$$p_k = \frac{e^{z_k}}{\sum_{j=0}^{C-1} e^{z_j}}$$

Every exponential is positive. The division by their sum makes the C probabilities add up to 1. The largest logit gets the largest probability. The cross-entropy loss of a row with the true class y is minus the logarithm of the probability of y:

$$\ell(z, y) = -\log p_y = -z_y + \log \sum_{j=0}^{C-1} e^{z_j}$$

$$L = \frac{1}{n} \sum_{i=1}^{n} \ell(z_i, y_i)$$

The second form follows from log(a / b) = log a − log b. `CrossEntropyLoss` returns L, the mean over the rows of a batch.

For the logits 2.0, 1.0, 0.1 and 0.0 with the true class 0, the steps give

$$\sum_j e^{z_j} = e^{2.0} + e^{1.0} + e^{0.1} + e^{0.0} = 12.2125$$

$$p_0 = e^{2.0}/12.2125 = 0.6050$$

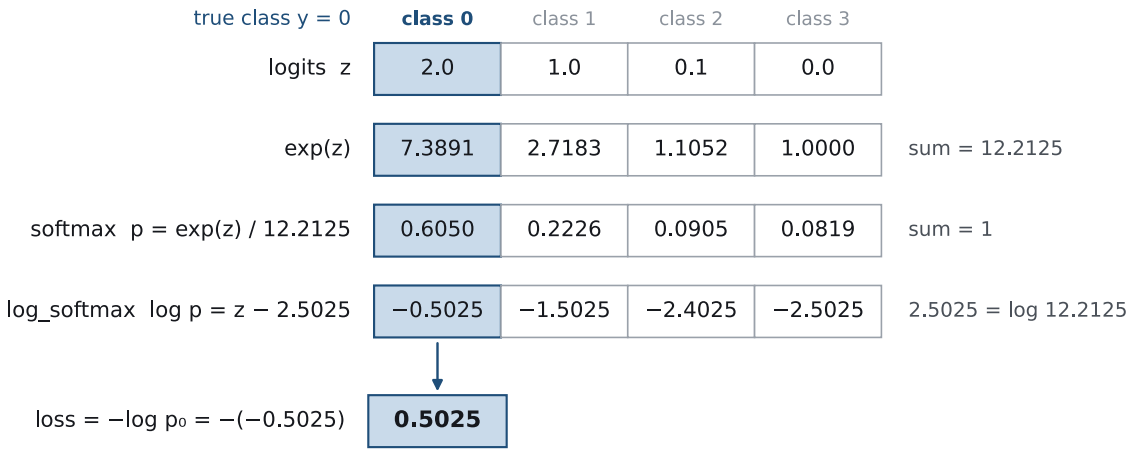
$$\log p_0 = z_0 - \log 12.2125 = 2.0 - 2.5025 = -0.5025$$

$$\ell(z, 0) = -\log p_0 = 0.5025$$

PyTorch computes the same steps:

```
y = torch.tensor([0])                          # the true class is 0
torch.exp(logits)                    # -> tensor([[7.3891, 2.7183, 1.1052, 1.0000]])
torch.exp(logits).sum().item()       # -> 12.2125
torch.softmax(logits, dim=1)         # -> tensor([[0.6050, 0.2226, 0.0905, 0.0819]])
torch.log_softmax(logits, dim=1)     # -> tensor([[-0.5025, -1.5025, -2.4025, -2.5025]])

nn.NLLLoss()(torch.log_softmax(logits, dim=1), y).item()    # -> 0.502461
nn.CrossEntropyLoss()(logits, y).item()                     # -> 0.502461   the same thing
```




The steps of CrossEntropyLoss for the logits above, one row per step. The marked column is the true class y = 0. Dividing by the sum, 12.2125, gives the probabilities. Subtracting log 12.2125 = 2.5025 from the logits gives their logarithms directly.

The softmax divides every exponential by the same sum. In logarithms the division becomes a subtraction. `torch.log_softmax` therefore subtracts log 12.2125 = 2.5025 from every logit. Its results are the logarithms of the softmax probabilities. `nn.NLLLoss`, the negative log-likelihood loss, takes such log-probabilities x and the class indices y. It returns minus the mean of the log-probabilities at the true classes:

$$\log p_k = z_k - \log \sum_{j=0}^{C-1} e^{z_j}$$

$$\mathrm{NLL}(x, y) = -\frac{1}{n} \sum_{i=1}^{n} x_{i, y_i}$$

`CrossEntropyLoss` is `log_softmax` followed by `NLLLoss`. It takes the logits, not probabilities. A softmax applied before it is therefore applied twice:

```
nn.CrossEntropyLoss()(torch.softmax(logits, dim=1), y).item()  # -> 1.055218   no error
```

No error or warning is raised. The model is then trained on a different loss.

### ARGMAX AND THE STARTING LOSS

```
torch.manual_seed(42)
net = TierNet(17)
loss_fn(net(xf), yf).item()              # -> 1.3969   untrained, against ln 4 = 1.3863
```

The softmax keeps the order of the logits in a row. `argmax` of the logits and `argmax` of the softmax are therefore the same class. The softmax is needed only for the probabilities themselves. On C balanced classes an untrained model starts near ln C, the loss of giving every class the probability 1 / C. This one starts at 1.3969, against ln 4 = 1.3863.

### VALIDATION AFTER EVERY EPOCH

```
torch.set_num_threads(4)                          # 4 threads, as for the values below

def train_epochs(net, opt, epochs):
    loader = DataLoader(TensorDataset(xf, yf), batch_size=128, shuffle=True)
    history = []
    for epoch in range(epochs):
        net.train()
        losses = []
        for xb, yb in loader:                     # 23 batches, in a new order every epoch
            opt.zero_grad()
            loss = loss_fn(net(xb), yb)
            loss.backward()
            opt.step()
            losses.append(loss.item())
        net.eval()
        with torch.no_grad():
            out = net(xv)                         # (732, 4): the validation rows
        history.append((np.mean(losses), loss_fn(out, yv).item(),
                        (out.argmax(dim=1) == yv).float().mean().item()))
    return np.array(history)    # (epochs, 3): training loss, validation loss and accuracy

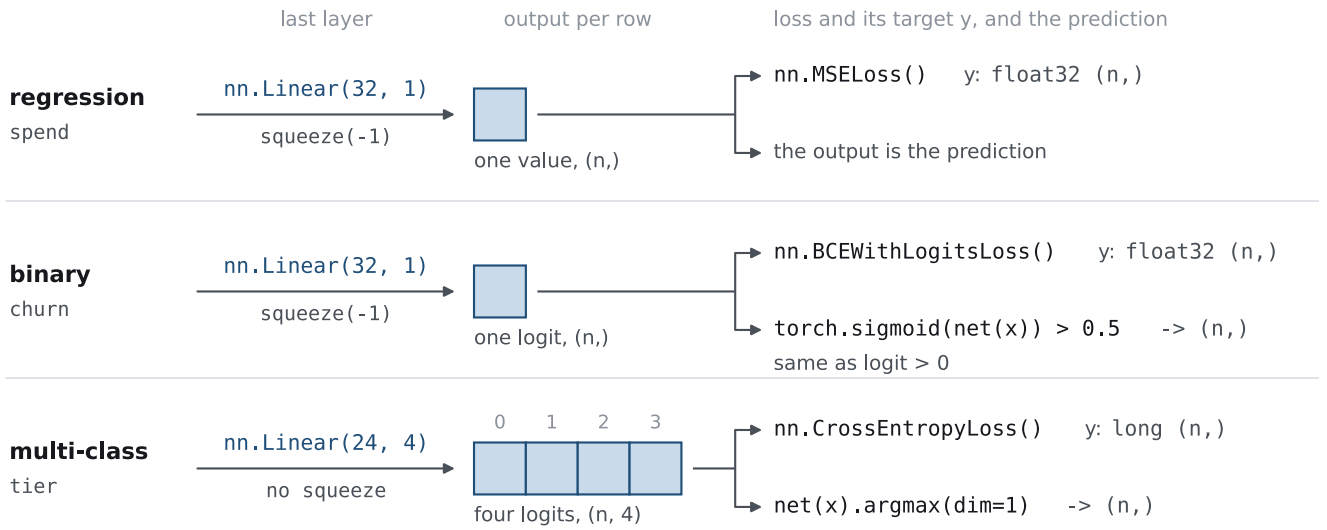
len(DataLoader(TensorDataset(xf, yf), batch_size=128))     # -> 23   batches per epoch
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
h = train_epochs(net, opt, 40)
best = int(h[:, 1].argmin())
best + 1, h[best].round(4).tolist()     # -> (21, [0.9152, 0.9405, 0.5546])   the best epoch
h[-1].round(4).tolist()                 # -> [0.8996, 0.9482, 0.5505]   after epoch 40
```

`TensorDataset(xf, yf)` returns the pair `(xf[i], yf[i])` for an index i. `train_epochs` draws its minibatches of 128 such pairs from a `DataLoader`, in a new order every epoch. It records three numbers after every epoch. They are the mean loss of the epoch's batches, the loss on the validation rows and the share of the validation rows whose largest logit is at the true tier. The validation rows are never used for a gradient step. During training they measure the loss and the accuracy on rows the network is not fitted on. Here the validation loss is lowest after epoch 21, at 0.9405. The network then classifies 0.555 of the validation rows correctly, against 0.25 for a constant answer or a random guess. After epoch 40 the validation loss is 0.948. A choice such as the number of epochs is made from these values. The test rows play no part in it.

`torch.set_num_threads(4)` fixes how many threads PyTorch uses on the CPU. The values below were computed with 4 threads. With another number of threads, the floating-point sums are rounded differently. A few of the values then differ. PyTorch also does not guarantee identical results across platforms.

### OUTPUT LAYER, LOSS AND LABEL DTYPE

The output layer of a network, together with its loss, is called the **head**. A network that predicts an amount, one that predicts a yes-or-no label and this one differ in the head, in the dtype of the target and in how the output is read as a prediction. `BCEWithLogitsLoss` takes one logit per row and a float label of 0.0 or 1.0. `CrossEntropyLoss` takes one logit per class and the class index as a `long`.




The three heads. Each lane shows the last layer, the output for one row, the loss with the dtype and shape of its target and how the output is read as a prediction.

02

## Per-class metrics

The 1200 rows of `audit_log.csv` hold the true tier of each row and the tier predicted by a classifier. The classifier is 78.5 per cent accurate but never predicts one of the four classes. From these rows we compute precision, recall and F1 for each class, to see how well the classifier does on every class separately.

```
with open("audit_log.csv", newline="") as f:
    log = list(csv.DictReader(f))       # 1200 rows: the true and the predicted tier
truth = np.array([TIERS.index(r["truth"]) for r in log])      # (1200,) true class
pred = np.array([TIERS.index(r["predicted"]) for r in log])   # (1200,) predicted class

(truth == pred).mean()          # -> 0.785

np.bincount(truth, minlength=4) # -> [387 509 239  65]   count per true class
np.bincount(pred, minlength=4)  # -> [404 565 231   0]   count per predicted class
```

The last entries differ. None of the 65 platinum rows is predicted as platinum. Platinum is 5.4 per cent of the rows. Missing all of them lowers the accuracy by only 5.4 points. The accuracy alone does not reveal it.

### THE CONFUSION MATRIX

A **confusion matrix** counts the rows by their true class and their predicted class.

```
cm = np.zeros((4, 4), dtype=int)
np.add.at(cm, (truth, pred), 1)     # cm[t, p] counts the rows of tier t predicted as tier p
cm
# -> [[334  29  24   0]
#     [ 24 461  24   0]
#     [ 46  46 147   0]
#     [  0  29  36   0]]
```

Row t of `cm` holds the rows of tier t. Column p holds the rows predicted as tier p. The diagonal holds the correct predictions. `np.add.at(cm, (truth, pred), 1)` adds 1 to `cm[truth[i], pred[i]]` for every row i. The plain `cm[truth, pred] += 1` would add 1 only once for every repeated pair of indices.

| | predicted | | | | |
|---|---|---|---|---|---|
| true | bronze | silver | gold | platinum | row total |
| bronze | 334 | 29 | 24 | 0 | 387 |
| silver | 24 | 461 | 24 | 0 | 509 |
| gold | 46 | 46 | **147** | 0 | **239** |
| platinum | 0 | 29 | 36 | 0 | 65 |
| column total | 404 | 565 | **231** | 0 | |

**for gold:**

tp = 147

fp = 84 (rest of the column)

fn = 92 (rest of the row)

precision = 147 / 231 = 0.636

recall = 147 / 239 = 0.615

The confusion matrix of the 1200 rows, true class by row and predicted class by column. For gold, precision divides tp = 147 by the column total 231. Recall divides it by the row total 239.

### PRECISION, RECALL AND F1

For one class c, three counts come from the matrix. The **true positives** tp are the rows of class c predicted as c, the diagonal cell `cm[c, c]`. The **false positives** fp are the rows of the other classes predicted as c, the rest of column c. The **false negatives** fn are the rows of class c predicted as another class, the rest of row c. Precision, recall and F1 combine them:

$$\text{precision} = \frac{tp}{tp + fp}, \qquad \text{recall} = \frac{tp}{tp + fn}$$

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2\,tp}{2\,tp + fp + fn}$$

Precision is the share of the rows predicted as c that belong to class c. Recall is the share of the rows of class c that the model predicts as c. A model that predicts c for only a few rows can have a high precision and a low recall. A model that predicts c for every row has a recall of 1. Its precision is then the share of class c among all rows. F1 is the harmonic mean of precision and recall. It is low whenever either of them is low. For gold:

```
c = TIERS.index("gold")                       # 2
tp = int(cm[c, c])                            # gold rows predicted as gold
fp = int(cm[:, c].sum()) - tp                 # rest of column c: predicted gold, not gold
fn = int(cm[c, :].sum()) - tp                 # rest of row c: gold, predicted as another tier
tp, fp, fn                                    # -> (147, 84, 92)

precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)
round(precision, 3), round(recall, 3), round(f1, 3)     # -> (0.636, 0.615, 0.626)
```

$$\text{precision} = \frac{147}{147 + 84} = 0.636, \qquad \text{recall} = \frac{147}{147 + 92} = 0.615$$

$$F_1 = \frac{2 \cdot 147}{2 \cdot 147 + 84 + 92} = \frac{294}{470} = 0.626$$

Of the 231 rows predicted as gold, 147 are gold. Of the 239 gold rows, 147 are predicted as gold. The table gives the four classes:

| CLASS | N | TP | FP | FN | PRECISION | RECALL | F1 |
|---|---|---|---|---|---|---|---|
| bronze | 387 | 334 | 70 | 53 | 0.827 | 0.863 | 0.845 |
| silver | 509 | 461 | 104 | 48 | 0.816 | 0.906 | 0.858 |
| gold | 239 | 147 | 84 | 92 | 0.636 | 0.615 | 0.626 |
| platinum | 65 | 0 | 0 | 65 | 0.000 | 0.000 | 0.000 |

Bronze and silver reach an F1 of 0.84 and 0.86. Gold reaches 0.63. The model never predicts platinum. Its tp + fp is therefore 0. Its precision would divide 0 by 0. The table gives it as 0. Its recall and its F1 are 0. The accuracy of 78.5 per cent does not show these differences.

#### CLASS IMBALANCE

The loss is averaged over rows. A class with 5.4 per cent of the rows contributes 5.4 per cent of the terms in that average. Predicting only the common classes keeps the average loss low and gets every row of the rare class wrong. The loss can be weighted by inverse class frequency with `nn.CrossEntropyLoss(weight=...)`. The training set can be resampled towards balance. Otherwise the per-class numbers are reported beside the accuracy of 78.5 per cent.

`np.bincount(pred)` next to an accuracy shows any class that is predicted rarely or never. For a binary classifier it shows when one class is predicted for every row.

---

03

## Feature scaling

We run the same gradient descent twice on two columns, `income` and `tenure_months`, once on the raw values and once on standardised ones. The two columns lie on very different scales. We compare the two runs to see how unscaled features affect gradient descent.

Both runs fit a logistic regression of churn on `income` and `tenure_months`, on all 3660 rows of `train.csv`. Each run takes 400 steps of plain gradient descent from zero weights, at a learning rate of 0.05.

```
x2 = np.stack([column(train, "income"), column(train, "tenure_months")], axis=1)
x2 = np.where(np.isnan(x2), np.nanmedian(x2, axis=0), x2)   # (3660, 2), gaps filled
churn = np.array([float(r["churn"]) for r in train])         # 1 for a customer who left
x2.min(axis=0).tolist(), x2.max(axis=0).tolist()
# -> ([6138.67, 0.0], [142610.65, 145.0])
x2.std(axis=0).round(1).tolist()                             # -> [14701.4, 19.3]
int(churn.sum()), int(len(churn) - churn.sum())              # -> (1385, 2275)   left, stayed

def descend(x, steps=400, lr=0.05):
    w, b, losses, weights, shares = np.zeros(2), 0.0, [], [], []
    for step in range(steps):
        p = 1 / (1 + np.exp(-np.clip(x @ w + b, -500, 500)))    # (3660,) probabilities
        loss = churn * np.log(p + 1e-12) + (1 - churn) * np.log(1 - p + 1e-12)
        losses.append(-loss.mean())
        weights.append(w.copy())
        shares.append((p > 0.5).mean())                          # predicted to churn
        w = w - lr * x.T @ (p - churn) / len(x)                 # a step down the gradient
        b = b - lr * (p - churn).mean()
    return np.array(losses), np.array(weights), np.array(shares)

raw_loss, raw_w, raw_share = descend(x2)
std_loss, std_w, std_share = descend((x2 - x2.mean(axis=0)) / x2.std(axis=0))
for k in (0, 5, 10, 20, 40, 80, 140, 220, 310, 399):
    print(f"step {k:>3}   unscaled {raw_loss[k]:6.3f}   standardised {std_loss[k]:.3f}")
# step   0   unscaled  0.693   standardised 0.693
# step   5   unscaled 17.175   standardised 0.684
# step  10   unscaled 10.456   standardised 0.677
# step  20   unscaled 10.456   standardised 0.664
# step  40   unscaled 17.175   standardised 0.647
# step  80   unscaled 10.456   standardised 0.630
# step 140   unscaled 10.456   standardised 0.620
# step 220   unscaled 10.456   standardised 0.617
# step 310   unscaled 10.456   standardised 0.616
# step 399   unscaled 17.175   standardised 0.616
```

`income` spans 6139 to 142611. `tenure_months` spans 0 to 145. Their standard deviations are 14701 and 19.3, a ratio of about 760. `descend` computes the predicted probabilities of all rows. It records the loss, the two weights and the share of the rows predicted to churn. It then takes one step of gradient descent. The loss is the mean binary cross-entropy with 1e-12 added inside each logarithm. `np.clip` keeps the exponential finite for the very large logits of the unscaled run. The second run standardises the two columns first.

The line that updates `w` subtracts the learning rate times the gradient of the mean loss L with respect to the weights. Row i has the inputs $x_i$, the label $y_i$, the logit $z_i = x_i \cdot w + b$ and the probability $p_i = \sigma(z_i)$. Its loss $\ell_i$ depends on the weight $w_j$ along the path $w_j \to z_i \to p_i \to \ell_i$. The chain rule multiplies one local derivative for each step of that path. The local derivatives are

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i} = \frac{p_i - y_i}{p_i\,(1 - p_i)}$$

$$\frac{\partial p_i}{\partial z_i} = \frac{e^{-z_i}}{\left(1 + e^{-z_i}\right)^2} = \frac{1}{1 + e^{-z_i}} \cdot \frac{e^{-z_i}}{1 + e^{-z_i}} = p_i\,(1 - p_i)$$

$$\frac{\partial z_i}{\partial w_j} = x_{i,j}, \qquad \frac{\partial z_i}{\partial b} = 1$$

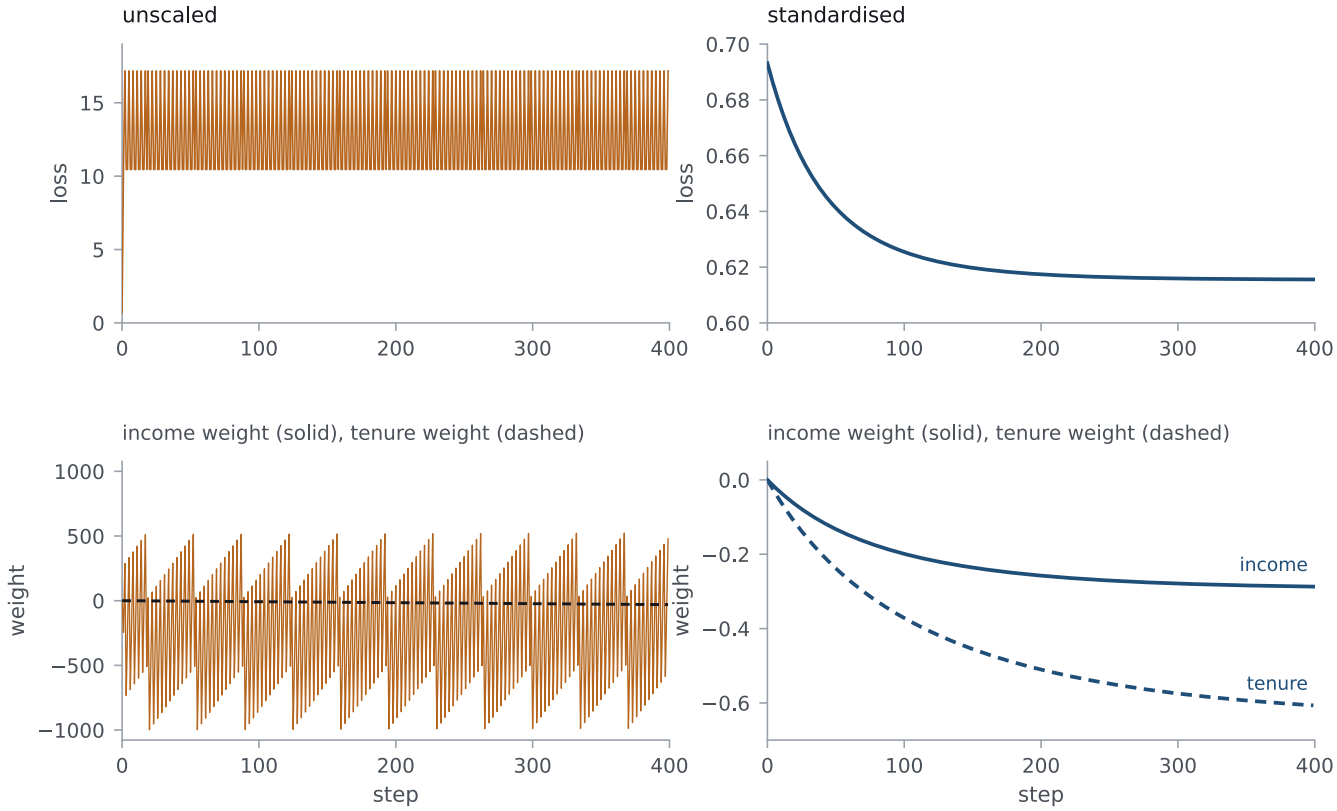
The derivative of log p is 1 / p. The derivative of log(1 − p) is −1 / (1 − p). The derivative of the sigmoid splits into two factors. The first is σ(z) and the second is 1 − σ(z). The logit is a sum of products. Its derivative

with respect to $w_j$ is the input that $w_j$ multiplies, $x_{i,j}$. Its derivative with respect to b is 1. The product along the path and the mean over the n rows give

$$\frac{\partial \ell_i}{\partial w_j} = \frac{\partial \ell_i}{\partial p_i}\frac{\partial p_i}{\partial z_i}\frac{\partial z_i}{\partial w_j} = \frac{p_i - y_i}{p_i\,(1-p_i)} \cdot p_i\,(1-p_i) \cdot x_{i,j} = (p_i - y_i)\,x_{i,j}$$

$$\frac{\partial L}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\,x_{i,j}, \qquad \frac{\partial L}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)$$

The factor $p_i(1 - p_i)$ cancels. Each row adds its error $p_i - y_i$ times its input. `p - churn` holds the errors of all 3660 rows. Entry j of `x.T @ (p - churn)` is the sum over the rows of column j of `x` times the errors. Divided by `len(x)`, its two entries are the derivatives of L with respect to the two weights. `(p - churn).mean()` is the derivative with respect to b. A positive derivative means that L grows with the parameter. A small step against the derivative therefore lowers L. The derivative of a weight is large when the column it multiplies holds large values.




The loss (top) and the weights (bottom) at each of the 400 steps, each panel on its own y axis. From step 1 on the unscaled loss takes only two values, 10.456 and 17.175. The unscaled income weight takes values between −998 and +523. The unscaled tenure weight drifts to −30.1.

Both runs start at 0.693. That is ln 2, the loss of a model that predicts a probability of 0.5 for every row. The standardised run then descends smoothly to 0.616 and stays there. The unscaled run jumps to 10.456 at step 1. From then on it takes only the values 10.456 and 17.175.

The two values are the losses of a model whose probabilities all lie within 1e-12 of 0 or of 1. A row whose true label gets the probability 0 adds $-\log(1\text{e-}12) = 27.631$ to the sum. At 17.175 every customer is predicted to churn. Each of the 2275 customers who stayed then adds 27.631. The mean over the 3660 rows is $27.631 \times 2275 / 3660 = 17.175$. At 10.456 no customer is predicted to churn. Each of the 1385 customers who left then adds 27.631. The mean is $27.631 \times 1385 / 3660 = 10.456$.

#### THE WEIGHTS DURING DESCENT

```
for k in range(7):
    print(f"step {k}   income {raw_w[k, 0]:7.1f}   tenure {raw_w[k, 1]:6.3f}   "
          f"predicted to churn {raw_share[k]:>4.0%}   loss {raw_loss[k]:6.3f}")
# step 0   income     0.0   tenure  0.000   predicted to churn   0%   loss  0.693
# step 1   income  -244.0   tenure -0.287   predicted to churn   0%   loss 10.456
# step 2   income   288.5   tenure  0.096   predicted to churn 100%   loss 17.175
# step 3   income  -732.0   tenure -0.862   predicted to churn   0%   loss 10.456
# step 4   income  -199.5   tenure -0.479   predicted to churn   0%   loss 10.456
# step 5   income   332.9   tenure -0.095   predicted to churn 100%   loss 17.175
# step 6   income  -687.5   tenure -1.053   predicted to churn   0%   loss 10.456
for k in (1, 10, 100, 399):                 # the standardised run
    print(f"step {k:>3}   income {std_w[k, 0]:7.4f}   tenure {std_w[k, 1]:7.4f}")
# step   1   income -0.0038   tenure -0.0064
# step  10   income -0.0350   tenure -0.0607
# step 100   income -0.1990   tenure -0.3718
# step 399   income -0.2871   tenure -0.6066
(x2.T @ (0.5 - churn) / len(x2)).round(2).tolist()          # -> [4880.14, 5.75]   gradients
round(x2[:, 0].mean())                                      # -> 31060   the mean income
inc = raw_w[1:, 0]                                          # income from step 1 on
round(float(inc.min()), 1), round(float(inc.max()), 1)      # -> (-997.7, 522.5)
int((np.diff(np.sign(inc)) != 0).sum())                     # -> 273   changes of sign
round(float(raw_w[399, 1]), 1)                              # -> -30.1   tenure at step 399
np.abs(np.diff(std_w, axis=0)).max().round(4)               # -> 0.0064   standardised
```

At step 0 every probability is 0.5. The gradient of the mean loss with respect to the income weight is the mean of $(p - y) \times$ income, 4880.1. One step at the learning rate 0.05 sets the weight to $-0.05 \times 4880.1 = -244.0$. A customer with the mean income, 31060, then gets the logit $-244.0 \times 31060$, about $-7.6$ million. Every probability becomes 0. The next gradient is large and negative. The step overshoots to $+288.5$. Every probability becomes 1. From then on the income weight takes values between $-998$ and $+523$. Its sign changes 273 times in the 398 steps from step 1 to step 399. The predictions jump with it, between no churn and churn for every customer. The tenure weight drifts slowly away from 0, to $-30.1$ at step 399. The standardised run moves both weights by less than 0.01 per step. At step 399 its weights are $-0.287$ for income and $-0.607$ for tenure.

#### GRADIENT SCALE AND LEARNING RATE

One learning rate is applied to every parameter. At the first step the gradient with respect to the income weight is 4880.1. The gradient for the tenure weight is 5.75, about 850 times smaller, because the input it multiplies is about 760 times smaller. A step small enough to be stable for income is so small for tenure that the tenure weight barely changes. A step large enough for tenure overshoots on income. It lands further from the minimum than it started. The next step overshoots back. These overshoots are the jumps of the income weight.

Standardising gives every column mean 0 and standard deviation 1. The gradients of the two weights are then of similar size. One learning rate suits both.

---

04

## Optimizers

Once `loss.backward()` has stored the gradients in `.grad`, `opt.step()` changes every parameter according to the rule of its optimizer. How fast the training loss falls depends on that rule. We compare three rules. Plain gradient descent subtracts the learning rate times the gradient. Momentum also adds a fraction of the previous step. Adam divides the step of each parameter by the size of its recent gradients.

```
def after_steps(make_opt, grad, steps=3):
    p = torch.zeros(2, requires_grad=True)
    opt = make_opt([p])
    out = []
    for step in range(steps):
        p.grad = grad.clone()               # in place of loss.backward()
        opt.step()
        out.append(p.tolist())
    return out                              # the parameter after every step

g = torch.tensor([1.0, 0.01])               # two parameters, gradients 100 times apart
after_steps(lambda ps: torch.optim.SGD(ps, lr=0.1), g)
# -> [[-0.1, -0.001], [-0.2, -0.002], [-0.3, -0.003]]
after_steps(lambda ps: torch.optim.SGD(ps, lr=0.1, momentum=0.9), g)
# -> [[-0.1, -0.001], [-0.29, -0.0029], [-0.561, -0.00561]]
after_steps(lambda ps: torch.optim.Adam(ps, lr=0.1), g)
# -> [[-0.1, -0.1], [-0.2, -0.2], [-0.3, -0.3]]
```

`after_steps` sets `p.grad` by hand to the same gradient at every step. Only the update rule then differs between the three runs. Each optimizer applies its rule to every parameter separately. The equations write $\theta_t$ for a parameter after step t, $g_t$ for its gradient at step t and $\eta$ for the learning rate.

#### SGD AND MOMENTUM

SGD subtracts the learning rate times the gradient:

$$\theta_t = \theta_{t-1} - \eta\, g_t$$

With $\eta = 0.1$, the first parameter, with the gradient 1.0, moves by 0.1 per step:

$$\theta_1 = 0 - 0.1 \cdot 1.0 = -0.1, \qquad \theta_2 = -0.1 - 0.1 \cdot 1.0 = -0.2, \qquad \theta_3 = -0.2 - 0.1 \cdot 1.0 = -0.3$$

The second parameter, with the gradient 0.01, moves by 0.001 per step. SGD with momentum adds every gradient to a buffer b. The buffer keeps the share $\mu$ of its previous value. The step is $\eta$ times the buffer:

$$b_1 = g_1, \qquad b_t = \mu\, b_{t-1} + g_t, \qquad \theta_t = \theta_{t-1} - \eta\, b_t$$

With $\mu = 0.9$, for the first parameter:

$$\begin{aligned} b_1 &= 1.0, & \theta_1 &= 0 - 0.1 \cdot 1.0 = -0.1 \\ b_2 &= 0.9 \cdot 1.0 + 1.0 = 1.9, & \theta_2 &= -0.1 - 0.1 \cdot 1.9 = -0.29 \\ b_3 &= 0.9 \cdot 1.9 + 1.0 = 2.71, & \theta_3 &= -0.29 - 0.1 \cdot 2.71 = -0.561 \end{aligned}$$

Under a gradient that keeps its sign, b grows towards $g / (1 - \mu) = 10\, g$. The step then grows towards ten times the plain one. A gradient that changes sign from step to step partly cancels in b.

#### ADAM

Adam keeps two running averages for every parameter, m of the gradient and v of its square. Both start at 0. The divisions by $1 - \beta_1{}^t$ and $1 - \beta_2{}^t$ correct for that start:

$$m_t = \beta_1\, m_{t-1} + (1 - \beta_1)\, g_t, \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$v_t = \beta_2\, v_{t-1} + (1 - \beta_2)\, g_t^2, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

The defaults are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$. Starting from $m_0 = 0$, t updates give every gradient so far a weight in $m_t$:

$$m_t = (1-\beta_1)\left(g_t + \beta_1 g_{t-1} + \beta_1^2 g_{t-2} + \cdots + \beta_1^{t-1} g_1\right) = (1-\beta_1)\sum_{k=1}^{t} \beta_1^{t-k} g_k$$

$$(1-\beta_1)\left(1 + \beta_1 + \beta_1^2 + \cdots + \beta_1^{t-1}\right) = 1 - \beta_1^t$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} = \frac{\sum_{k=1}^{t} \beta_1^{t-k} g_k}{\sum_{k=1}^{t} \beta_1^{t-k}}$$

The newest gradient has the weight $1 - \beta_1$. Each step further back multiplies the weight by $\beta_1$. With $\beta_1 = 0.9$, a gradient 10 steps old keeps $0.9^{10} = 0.35$ of the weight of the newest one. m therefore follows the recent gradients. The factor $1 - \beta_1$ keeps m on the scale of the gradient. The momentum buffer b adds every gradient in full and grows towards 10 g.

Multiplied out, the second line keeps only 1 and $-\beta_1{}^t$. The other terms cancel in pairs. The weights therefore add up to $1 - \beta_1{}^t$, not to 1. The weights of the steps before step 1 are missing, because m starts at 0. In the first steps $m_t$ is therefore too small. Dividing by the sum of the weights makes $\hat{m}_t$ a weighted average whose weights add up to 1. Under a constant gradient g it is exactly g. At step 1 the sum is $1 - 0.9 = 0.1$. As t grows, $\beta_1{}^t$ goes to 0 and the correction fades. $v_t$ and $\hat{v}_t$ are built the same way from the squared gradients. With $\beta_2 = 0.999$ the weights shrink by 0.1 per cent per step rather than 10 per cent. v therefore averages over many more steps than m.

$\sqrt{\hat{v}_t}$ is the square root of an average of squared gradients. It measures the size of the recent gradients, in the units of the gradient. $\hat{m}_t$ divided by it has no units. Multiplying every gradient of a parameter by 100 multiplies both $\hat{m}_t$ and $\sqrt{\hat{v}_t}$ by 100 and leaves the step unchanged. Gradients that change sign from step to step cancel in $\hat{m}_t$. They do not cancel in $\hat{v}_t$, whose terms are all positive. The ratio is then small and the step shrinks. $\varepsilon$ keeps the division finite when every gradient so far is 0.

For the second parameter, with $\eta = 0.1$ and the gradient 0.01, the first step is

$$m_1 = 0.1 \cdot 0.01 = 0.001, \qquad \hat{m}_1 = 0.001/0.1 = 0.01$$
$$v_1 = 0.001 \cdot 0.01^2 = 10^{-7}, \qquad \hat{v}_1 = 10^{-7}/0.001 = 10^{-4}$$
$$\theta_1 = 0 - 0.1 \cdot \frac{0.01}{\sqrt{10^{-4}} + 10^{-8}} = -0.1$$

The second step is

$$m_2 = 0.9 \cdot 0.001 + 0.1 \cdot 0.01 = 0.0019, \qquad \hat{m}_2 = 0.0019/0.19 = 0.01$$
$$v_2 = 0.999 \cdot 10^{-7} + 0.001 \cdot 10^{-4} = 1.999 \cdot 10^{-7}, \qquad \hat{v}_2 = 1.999 \cdot 10^{-7}/0.001999 = 10^{-4}$$
$$\theta_2 = -0.1 - 0.1 \cdot \frac{0.01}{\sqrt{10^{-4}} + 10^{-8}} = -0.2$$

Under a constant gradient, the corrected average of the gradient equals the gradient. The square root of the corrected average of its square equals the size of the gradient. Their ratio is 1 or $-1$. Every parameter then moves by the learning rate per step, whatever the size of its gradient. The two parameters, with gradients 100 times apart, both move by 0.1.

### PATHS ON AN ELONGATED LOSS

The function `bowl` below is a loss of two parameters:

$$L(w) = \tfrac{1}{2}(w_0^2 + 25\,w_1^2), \qquad \nabla L(w) = (w_0,\ 25\,w_1)$$

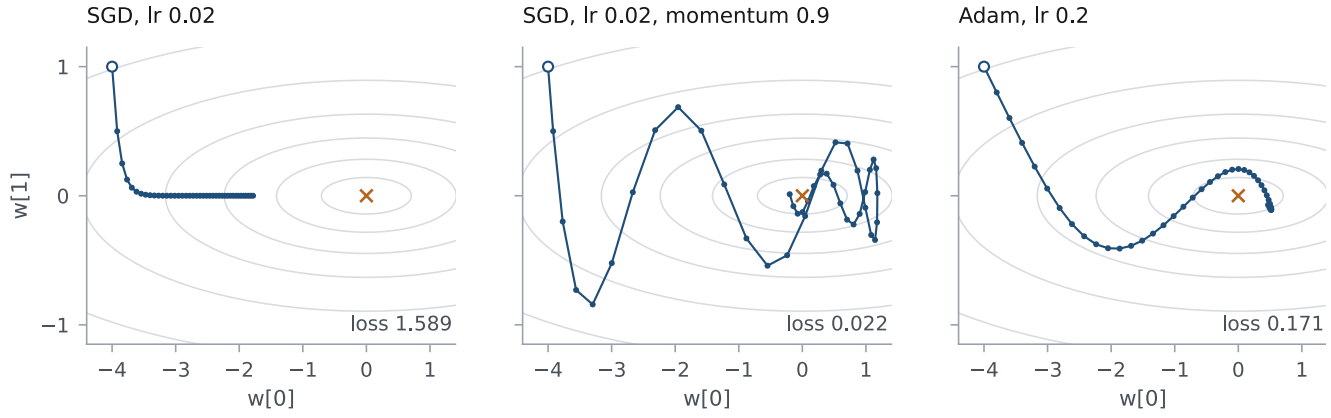
Its minimum is 0, at $w = (0, 0)$. It is 25 times as curved along $w_1$ as along $w_0$. Its contour lines are long ellipses. They form a valley along $w_0$. A loss on two features of different scales takes this shape. Each of the three optimizers starts at $w = (-4, 1)$ and takes 40 steps.

```
def bowl(w):                                    # its minimum is 0, at w = (0, 0)
    return 0.5 * (w[0] ** 2 + 25 * w[1] ** 2)   # 25 times as curved along w[1]

def path(make_opt, steps=40):
    w = torch.tensor([-4.0, 1.0], requires_grad=True)
    opt = make_opt([w])
    points = [w.detach().clone()]
    for step in range(steps):
        opt.zero_grad()
        bowl(w).backward()
        opt.step()
        points.append(w.detach().clone())
    return torch.stack(points)                  # (41, 2): the start and every step
```

`path` runs one optimizer for 40 steps from $w = (-4, 1)$. It returns the start and the point after every step.

```
paths = {"SGD": path(lambda ps: torch.optim.SGD(ps, lr=0.02)),
         "momentum": path(lambda ps: torch.optim.SGD(ps, lr=0.02, momentum=0.9)),
         "Adam": path(lambda ps: torch.optim.Adam(ps, lr=0.2))}
{name: round(bowl(p[-1]).item(), 3) for name, p in paths.items()}
# -> {'SGD': 1.589, 'momentum': 0.022, 'Adam': 0.171}
paths["SGD"][-1]                            # -> tensor([-1.7828e+00,  9.0949e-13])
paths["Adam"][1] - paths["Adam"][0]         # -> tensor([ 0.2000, -0.2000])   Adam's first step
```




The 40 steps of each optimizer from w = (−4, 1), the open circle, on the contour lines of bowl, with w[1] drawn at about twice the scale of w[0]. The cross marks the minimum. Each panel gives the loss after the last step.

SGD at $\eta = 0.02$ multiplies each coordinate by a fixed factor at every step, because the gradient of bowl is ($w_0$, 25 $w_1$):

$$w_0 \leftarrow (1 - \eta)\,w_0 = 0.98\,w_0, \qquad w_1 \leftarrow (1 - 25\,\eta)\,w_1 = 0.5\,w_1$$

Across the valley, $w_1$ halves at every step. The path reaches the floor of the valley within a few steps. Along the valley, $w_0$ shrinks by 2 per cent per step. After 40 steps it is $-4 \times 0.98^{40} = -1.78$. The loss is still 1.589.

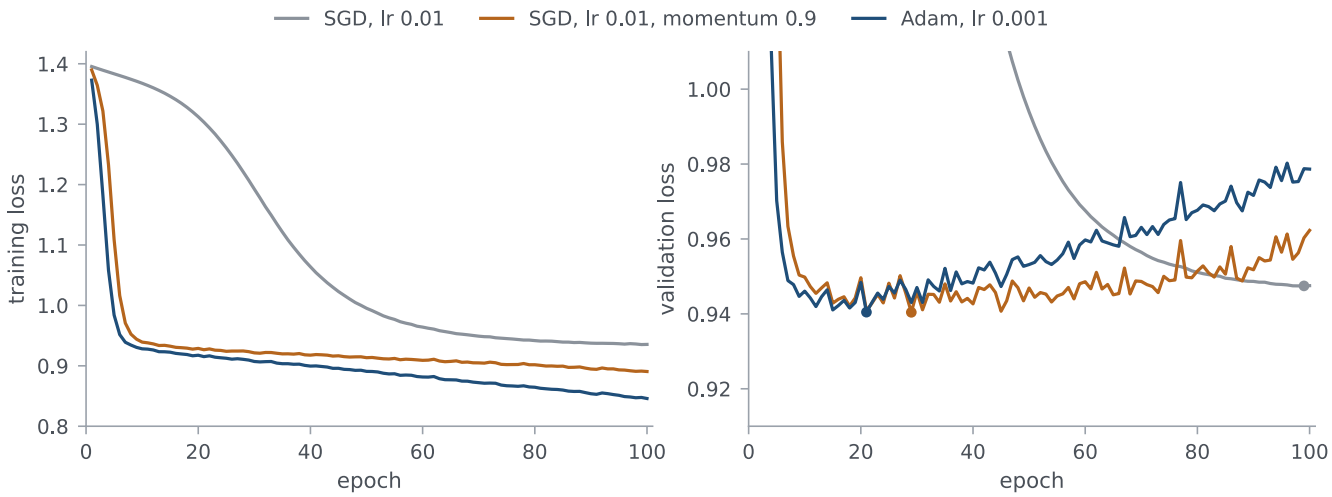
A learning rate above $2 / 25 = 0.08$ would make the factor $1 - 25\eta$ smaller than $-1$. The steps across the valley would then grow.

SGD with momentum 0.9 at the same $\eta = 0.02$ adds up the gradients along the valley in its buffer. Its steps along the valley grow towards ten times the plain step. The path first swings across the valley. The gradient across the valley changes sign from one swing to the next. It partly cancels in the buffer. The swings shrink. After 40 steps the loss is 0.022.

Adam at $\eta = 0.2$ moves both coordinates by about 0.2 per step at first, whatever the size of their gradients. Its first steps therefore run along the diagonal. After 40 steps the loss is 0.171.

#### SGD, MOMENTUM AND ADAM ON TIERNET

```
runs = {"SGD": lambda ps: torch.optim.SGD(ps, lr=0.01),
        "momentum": lambda ps: torch.optim.SGD(ps, lr=0.01, momentum=0.9),
        "Adam": lambda ps: torch.optim.Adam(ps, lr=0.001)}
curves = {}
for name, make in runs.items():
    torch.manual_seed(42)
    net = TierNet(17)
    curves[name] = h = train_epochs(net, make(net.parameters()), 100)
    best = h[:, 1].argmin()
    print(f"{name:<9} training {h[9, 0]:.3f} {h[99, 0]:.3f}   "
          f"validation best {h[best, 1]:.3f} at epoch {best + 1}, last {h[99, 1]:.3f}")
# SGD       training 1.368 0.935   validation best 0.947 at epoch 99, last 0.948
# momentum  training 0.939 0.891   validation best 0.940 at epoch 29, last 0.962
# Adam      training 0.928 0.846   validation best 0.940 at epoch 21, last 0.979
int(np.argmax(curves["SGD"][:, 0] <= curves["momentum"][9, 0])) + 1     # -> 86
bool((curves["SGD"][:, 0] <= curves["Adam"][9, 0]).any())               # -> False
{n: int(np.argmax(h[:, 1] < 1.01)) + 1 for n, h in curves.items()}
# -> {'SGD': 47, 'momentum': 6, 'Adam': 5}
```




The training loss (left) and the validation loss (right) of TierNet at every epoch of the three runs above, all from the same initial weights. The dots mark the lowest validation loss of each run. The right panel stops at 1.01. The validation loss comes below it at epoch 47 for SGD, at epoch 6 for momentum and at epoch 5 for Adam.

Momentum at the same learning rate brings the training loss to 0.939 in 10 epochs. Plain SGD needs 86 epochs for that. Adam at 0.001 reaches 0.928 in 10 epochs. Plain SGD does not reach 0.928 in 100 epochs. The lowest validation losses are close together, 0.947 for SGD and 0.940 for momentum and for Adam. They come at epochs 99, 29 and 21. After their minimum the validation losses of momentum and Adam rise, to 0.962 and 0.979 at epoch 100. Their training losses keep falling, to 0.891 and 0.846. The faster optimizers start to overfit sooner. Plain SGD reaches its lowest validation loss only at epoch 99.

### ADAM ON UNSCALED FEATURES

Adam does not remove the need to scale the features. A logistic regression of churn on the unscaled income and tenure_months is fitted as `nn.Linear(2, 1)` from zero weights. It uses `BCEWithLogitsLoss` on all 3660 rows and Adam at a learning rate of 0.05.

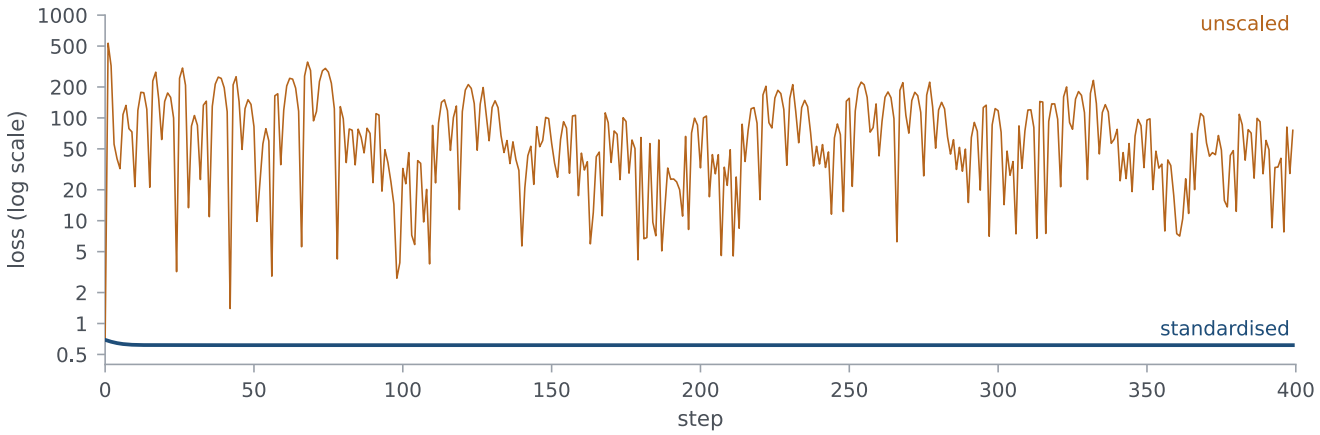
```
xu = torch.tensor(x2, dtype=torch.float32)          # the unscaled income and tenure_months
yu = torch.tensor(churn, dtype=torch.float32)

def adam_fit(x, steps):
    model = nn.Linear(2, 1)
    nn.init.zeros_(model.weight)
    nn.init.zeros_(model.bias)
    opt = torch.optim.Adam(model.parameters(), lr=0.05)
    losses = []
    for step in range(steps):
        opt.zero_grad()
        loss = nn.BCEWithLogitsLoss()(model(x).squeeze(1), yu)
        loss.backward()
        opt.step()
        losses.append(loss.item())
    return model, np.array(losses)

model, _ = adam_fit(xu, 1)
model.weight.grad.tolist()      # -> [[4880.14, 5.75]]   the first gradients
model.weight.tolist()           # -> [[-0.05, -0.05]]    after one step
round(0.05 * x2[:, 0].mean())   # -> 1553   the change of the logit at the mean income

_, losses = adam_fit(xu, 400)
round(losses[5:].min(), 3)                                  # -> 1.396   from step 5 on
round(float(losses[300:].min()), 1), round(float(losses[300:].max()), 1)  # -> (6.1, 208.0)
_, losses = adam_fit(torch.tensor((x2 - x2.mean(axis=0)) / x2.std(axis=0),
                                  dtype=torch.float32), 400)
round(losses[-1], 3)                                        # -> 0.615   standardised
```

`nn.init.zeros_` sets a tensor to 0 in place. The first gradients are 4880.1 for the income weight and 5.75 for the tenure weight. Adam moves both weights by 0.05 in its first step, whatever the size of their gradients. A change of 0.05 in the income weight changes the logit of a customer with the mean income, 31060, by 1553. The probabilities therefore swing between 0 and 1. From step 5 on the loss never falls below 1.396. Over the last 100 steps it lies between 6.1 and 208.0. On the standardised columns the same optimizer ends at 0.615.




The loss at every step of the two 400-step runs above, on a logarithmic axis. On the unscaled columns it jumps between about 1.4 and a few hundred until the last step. On the standardised columns it falls smoothly to 0.615.

05

## Regularisation

A network with many parameters can fit its training rows ever more closely. Its validation loss rises at the same time. This is **overfitting**. Weight decay and dropout limit how closely a network can fit its training rows. We try both on a wide network that overfits strongly without them.

### A WIDE NETWORK

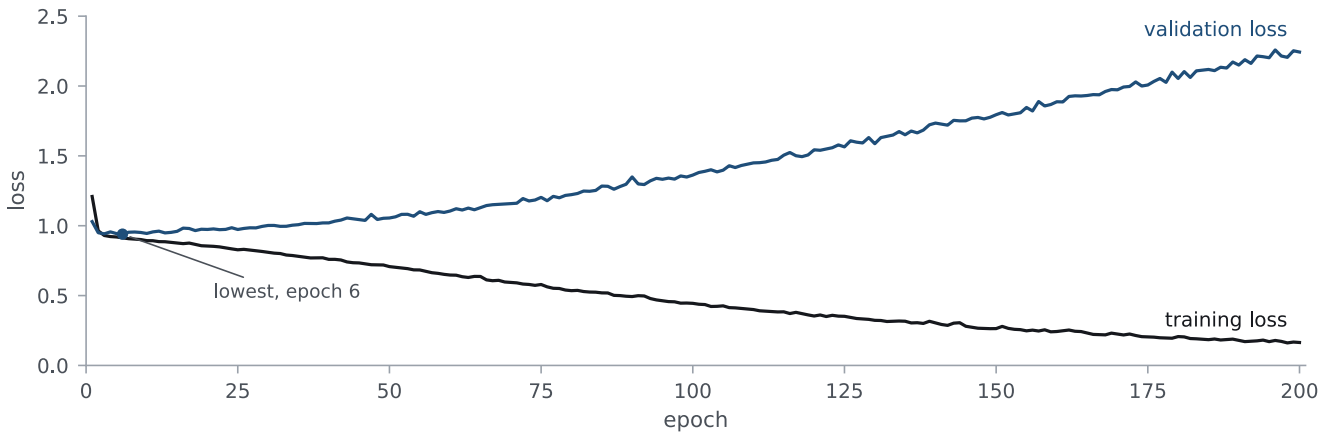
```
def wide():
    return nn.Sequential(nn.Linear(17, 256), nn.ReLU(),
                         nn.Linear(256, 128), nn.ReLU(),
                         nn.Linear(128, 4))

def run(make, decay=0.0):
    torch.manual_seed(42)                    # the same initial weights in every run
    net = make()
    opt = torch.optim.Adam(net.parameters(), lr=1e-3, weight_decay=decay)
    return net, train_epochs(net, opt, 200)

def report(name, h):
    best = h[:, 1].argmin()
    print(f"{name:<18} training {h[-1, 0]:.3f}   validation best {h[best, 1]:.3f} "
          f"at epoch {best + 1}, last {h[-1, 1]:.3f}")

sum(q.numel() for q in wide().parameters())         # -> 38020
sum(q.numel() for q in TierNet(17).parameters())    # -> 2140
net, h = run(wide)
report("none", h)
# none               training 0.160   validation best 0.940 at epoch 6, last 2.257
with torch.no_grad():
    fit_accuracy = (net(xf).argmax(dim=1) == yf).float().mean().item()
round(fit_accuracy, 3), round(float(h[-1, 2]), 3)   # -> (0.977, 0.448)   fitting, validation
```

`wide` has 38020 parameters, about 13 for every one of the 2928 fitting rows. `TierNet` has 2140. The count alone does not show whether a network has too many parameters for its data. The losses and accuracies on the two kinds of rows show it. Trained for 200 epochs with Adam at 0.001, `wide` reaches a training loss of 0.160. It then classifies 0.977 of the fitting rows correctly and 0.448 of the validation rows. Its validation loss is lowest at epoch 6, at 0.940. It rises to 2.257 by epoch 200. The network has fitted the particular fitting rows rather than the relationship between the features and the tier. The lowest validation loss of the wide network is about the same as that of TierNet. The extra parameters do not give a better network.




The training loss and the validation loss of the wide network at every epoch. The dot marks the lowest validation loss, 0.940 at epoch 6. After it the training loss keeps falling and the validation loss rises.

### WEIGHT DECAY

**Weight decay** adds $\lambda$ times each parameter to its gradient before the update. The update of SGD then becomes

$$\theta_t = \theta_{t-1} - \eta \left( \nabla L(\theta_{t-1}) + \lambda \, \theta_{t-1} \right) = (1 - \eta\lambda) \, \theta_{t-1} - \eta \, \nabla L(\theta_{t-1})$$

Every step multiplies each weight by $1 - \eta\lambda$ before the step of the loss gradient. A weight stays away from 0 only where the gradient of the loss moves it away by as much as the decay moves it back. In PyTorch $\lambda$ is the argument `weight_decay`.

```
p = torch.tensor([1.0, -2.0], requires_grad=True)
opt = torch.optim.SGD([p], lr=0.1, weight_decay=0.2)
p.grad = torch.zeros(2)                 # no gradient from the loss
opt.step()
p.tolist()                              # -> [0.98, -1.96]
p.grad = torch.tensor([0.5, 0.5])       # a gradient of 0.5 from the loss, for both weights
opt.step()
p.tolist()                              # -> [0.9104, -1.9708]
```

With $\eta = 0.1$ and $\lambda = 0.2$ the factor is $1 - 0.1 \times 0.2 = 0.98$:

step 1, $\nabla L = 0$: $\quad 0.98 \cdot 1.0 = 0.98, \qquad 0.98 \cdot (-2.0) = -1.96$

step 2, $\nabla L = 0.5$: $\quad 0.98 \cdot 0.98 - 0.1 \cdot 0.5 = 0.9104, \quad 0.98 \cdot (-1.96) - 0.1 \cdot 0.5 = -1.9708$

`Adam` with `weight_decay` adds $\lambda\theta$ to the gradient in the same way. The sum then goes through Adam's division by the size of the recent gradients. The decay of a weight then no longer depends on its size. `torch.optim.AdamW` subtracts $\eta\lambda\theta$ from each weight directly, outside that division:

$$\theta_t = \theta_{t-1} - \eta\lambda \, \theta_{t-1} - \eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

```
for make in (lambda ps: torch.optim.Adam(ps, lr=0.1, weight_decay=0.2),
             lambda ps: torch.optim.AdamW(ps, lr=0.1, weight_decay=0.2)):
    p = torch.tensor([1.0, -2.0], requires_grad=True)
    opt = make([p])
    p.grad = torch.zeros(2)             # again no gradient from the loss
    opt.step()
    print([round(v, 4) for v in p.tolist()])
# [0.9, -1.9]
# [0.98, -1.96]
```

Without a gradient from the loss, the gradient of Adam is $\lambda\theta$, 0.2 and $-0.4$. Divided by its own size, it moves both weights by the learning rate, 0.1. AdamW multiplies both weights by $1 - 0.1 \times 0.2 = 0.98$.

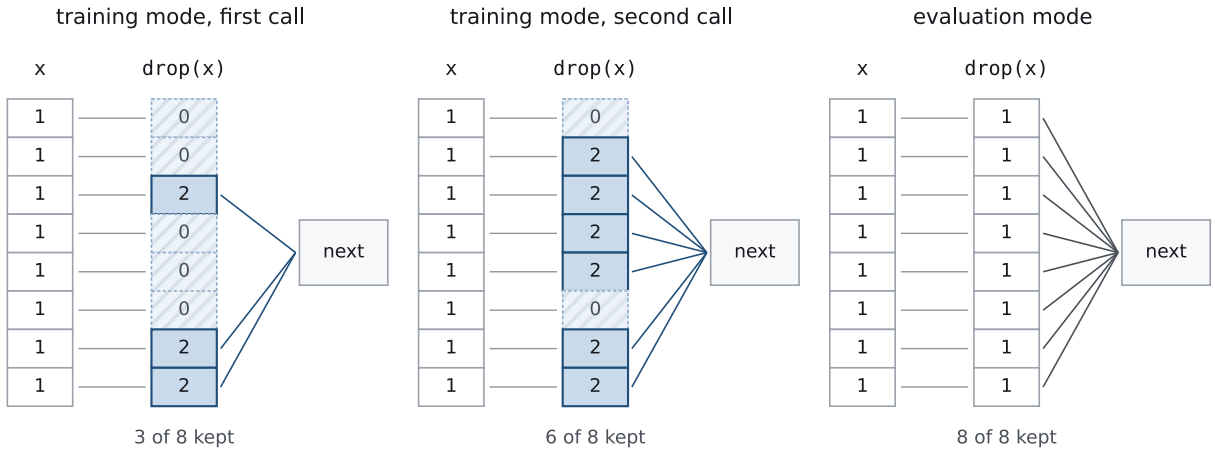
### DROPOUT

`nn.Dropout(p)` multiplies each element $x_i$ of its input by a random $m_i$ and divides by $1 - p$:

$$y_i = \frac{m_i \, x_i}{1 - p}, \qquad m_i = \begin{cases} 0 & \text{with probability } p \\ 1 & \text{otherwise} \end{cases}$$

```
drop = nn.Dropout(p=0.5)
torch.manual_seed(0)
drop(torch.ones(8))         # -> tensor([0., 0., 2., 0., 0., 0., 2., 2.])   training mode
drop(torch.ones(8))         # -> tensor([0., 2., 2., 2., 2., 0., 2., 2.])   a new pattern
drop.eval()
drop(torch.ones(8))         # -> tensor([1., 1., 1., 1., 1., 1., 1., 1.])   evaluation mode
```

Each $m_i$ is 0 with probability p, here 0.5. A new pattern is drawn on every call. The kept elements are divided by $1 - p = 0.5$, here doubled. The expected value of every element is then unchanged. In evaluation mode the layer passes its input through unchanged. With dropout after a hidden layer, the next layer is trained on a different random subset of about half of the units at every step.




The two training calls and the evaluation call above, for the 8 outputs x of a hidden layer with dropout after it. In a training call the hatched units are set to 0 and pass nothing to the next layer. The other units pass 2, their value divided by 1 − p. In evaluation mode every unit passes its value unchanged.

Once drawn, the mask is a fixed set of numbers. $y_i = m_i\, x_i / (1 - p)$ is then a product of $x_i$ with a constant, and backpropagation passes through it like through any other product. Its local derivative is $\partial y_i / \partial x_i = m_i / (1 - p)$:

```
x = torch.ones(8, requires_grad=True)
drop.train()                # back to training mode
torch.manual_seed(0)        # the first pattern above again
drop(x).sum().backward()
x.grad                      # -> tensor([0., 0., 2., 0., 0., 0., 2., 2.])
```

The gradient of the sum with respect to every $x_i$ is $m_i / (1 - p)$. It is 0 for a dropped element and 2 for a kept one. A dropped unit passes no gradient back, so the weights that compute it get nothing through it from that row. The layer keeps the mask of the forward pass for the backward pass. Drawing the mask is the only random part. The rest is an ordinary multiplication, so the chain rule applies to the network with dropout unchanged.

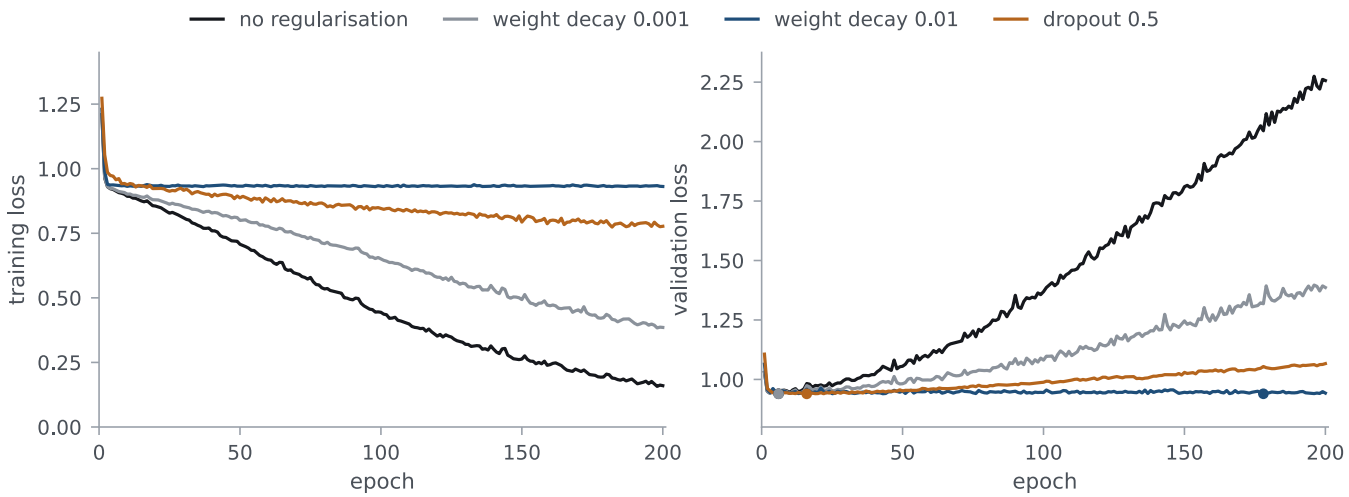
```
def wide_dropout(p):
    return nn.Sequential(nn.Linear(17, 256), nn.ReLU(), nn.Dropout(p),
                         nn.Linear(256, 128), nn.ReLU(), nn.Dropout(p),
                         nn.Linear(128, 4))

sum(q.numel() for q in wide_dropout(0.5).parameters())      # -> 38020   no parameters added
```

`wide_dropout(p)` places an `nn.Dropout(p)` after each hidden layer of `wide`. Dropout has no parameters. The network keeps its 38020. `train_epochs` calls `net.train()` before the batches of an epoch and `net.eval()` before the validation rows. The two calls switch every dropout layer between its two modes.

FOUR RUNS OF THE WIDE NETWORK

```
settings = [("weight decay 0.001", wide, 1e-3), ("weight decay 0.01", wide, 1e-2),
            ("dropout 0.5", lambda: wide_dropout(0.5), 0.0)]
regs = {"none": h}
for name, make, decay in settings:
    _, regs[name] = run(make, decay)
    report(name, regs[name])
# weight decay 0.001 training 0.385   validation best 0.940 at epoch 6, last 1.387
# weight decay 0.01  training 0.931   validation best 0.940 at epoch 178, last 0.943
# dropout 0.5        training 0.777   validation best 0.939 at epoch 16, last 1.067
round(float(regs["weight decay 0.01"][49, 1]), 3), round(float(regs["none"][49, 1]), 3)
# -> (0.951, 1.055)
```




The training loss (left) and the validation loss (right) of the wide network at every epoch of the four runs, all from the same initial weights. The training loss of the dropout run is the mean loss of its batches, computed with dropout active. The dots mark the lowest validation loss of each run.

Weight decay 0.01 holds the training loss at 0.931 after 200 epochs and the validation loss at 0.943. The validation loss of this run is lowest at epoch 178, at 0.940. Weight decay 0.001 and dropout 0.5 slow the rise of the validation loss without stopping it. At epoch 200 it is 1.387 and 1.067, against 2.257 without either. The lowest validation loss is about the same in all four runs, 0.939 to 0.940. On these customers neither method reaches a lower validation loss than early stopping does. With weight decay 0.01 the validation loss depends far less on the epoch at which training stops. It is 0.951 at epoch 50 and 0.943 at epoch 200, against 1.055 and 2.257 without regularisation.

---

06

## Model selection

Like the number of epochs, the depth, the width and the learning rate of a network are settings that we choose on the validation rows. We train each candidate with early stopping and pick the one with the lowest validation loss. We use the 1000 rows of `test.csv` only once, on the network we have chosen. Its accuracy on them then measures how well it does on new customers.

```
def mlp(depth, width):
    layers, n = [], 17
    for _ in range(depth):
        layers += [nn.Linear(n, width), nn.ReLU()]
        n = width
    return nn.Sequential(*layers, nn.Linear(n, 4))

def best_of(net, opt, patience=20, max_epochs=200):
    best_loss, best_epoch, best_state = float("inf"), 0, None
    for epoch in range(max_epochs):
        val_loss = float(train_epochs(net, opt, 1)[0, 1])
        if val_loss < best_loss:
            best_loss, best_epoch = val_loss, epoch
            best_state = copy.deepcopy(net.state_dict())
        elif epoch - best_epoch >= patience:
            break
    net.load_state_dict(best_state)         # the weights of the best epoch
    return best_loss
```

`mlp(depth, width)` builds a network of `depth` hidden layers of `width` units each, with ReLU after each. `best_of` trains a network with early stopping on the validation loss and a patience of 20 epochs. It restores the weights of its best epoch and returns that epoch's validation loss.

```
results = []
for depth, width, lr in itertools.product([1, 2, 3], [16, 64, 256], [1e-3, 1e-2]):
    torch.manual_seed(42)
    net = mlp(depth, width)
    loss = best_of(net, torch.optim.Adam(net.parameters(), lr=lr))
    results.append((loss, depth, width, lr, net))
results.sort(key=lambda r: r[0])                # the lowest validation loss first
for loss, depth, width, lr, net in results[:4] + results[-2:]:
    print(f"depth {depth}   width {width:>3}   lr {lr:<5}   validation loss {loss:.4f}")
# depth 1   width  16   lr 0.01    validation loss 0.9354
# depth 2   width  16   lr 0.01    validation loss 0.9383
# depth 1   width  64   lr 0.001   validation loss 0.9393
# depth 1   width  16   lr 0.001   validation loss 0.9401
# depth 2   width  64   lr 0.01    validation loss 0.9569
# depth 3   width 256   lr 0.01    validation loss 0.9637
```

`itertools.product` runs through the 18 combinations of three depths, three widths and two learning rates. Each combination is trained with Adam from the same seed. The loop prints the best four and the worst two. The networks with one hidden layer take three of the four best places. The two worst combine the learning rate 0.01 with two or three hidden layers of 64 or 256 units. The best four lie between 0.9354 and 0.9401.

#### VARIATION BETWEEN SEEDS

```
seed_losses = []
for seed in range(5):
    torch.manual_seed(seed)
    net = mlp(1, 16)
    seed_losses.append(best_of(net, torch.optim.Adam(net.parameters(), lr=1e-2)))
round(min(seed_losses), 4), round(max(seed_losses), 4)      # -> (0.9348, 0.9422)
round(results[8][0], 4)                                     # -> 0.9419   the ninth
```

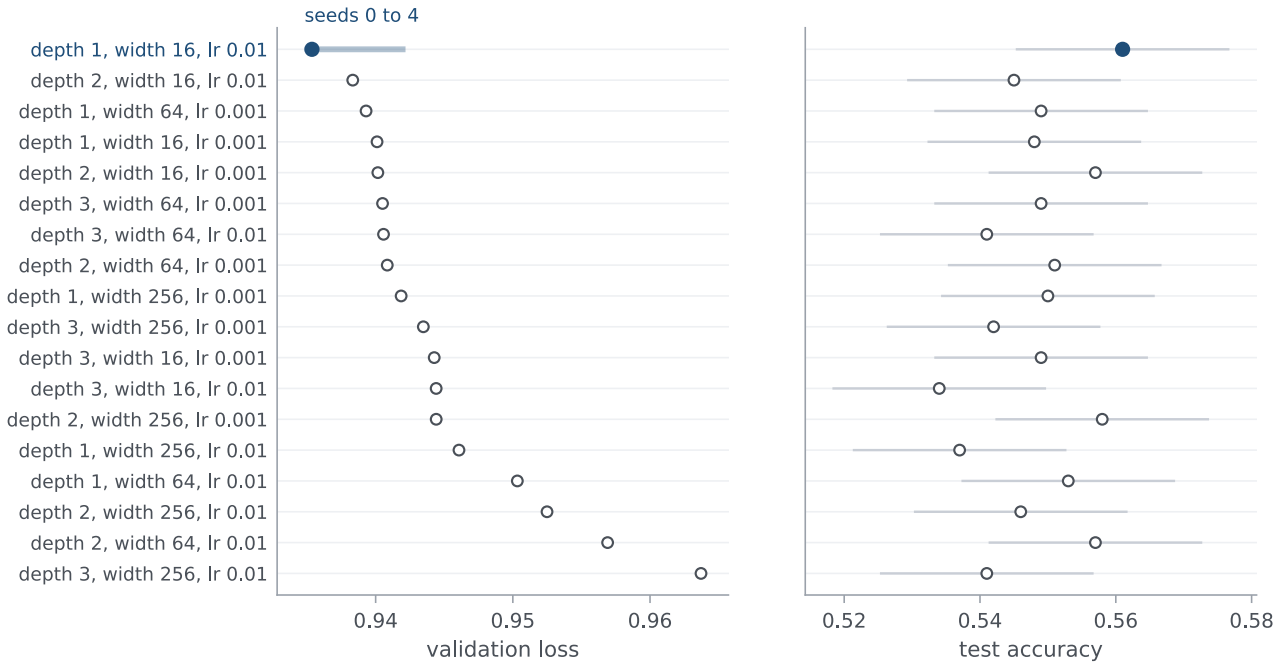
The configuration with the lowest validation loss has one hidden layer of 16 units and the learning rate 0.01. Trained from the seeds 0 to 4 instead of 42, it reaches validation losses from 0.9348 to 0.9422. That spread of 0.0074 is larger than the gap between the first and the ninth configuration, 0.9354 and 0.9419. Its first place therefore depends on the random initial weights and batches as well as on its depth, width and learning rate.

### THE TEST FILE

```
loss, depth, width, lr, net = results[0]
with torch.no_grad():
    accuracy = (net(xt).argmax(dim=1) == yt).float().mean().item()
    val_accuracy = (net(xv).argmax(dim=1) == yv).float().mean().item()
round(accuracy, 3), round(val_accuracy, 3)      # -> (0.561, 0.557)   test, validation
```

The chosen network classifies 0.561 of the 1000 rows of `test.csv` correctly. This is the number to report for it. Its accuracy on the validation rows is 0.557.

```
with torch.no_grad():
    accs = [(net(xt).argmax(dim=1) == yt).float().mean().item() for *_, net in results]
round(min(accs), 3), round(max(accs), 3)        # -> (0.534, 0.561)   all 18
[k + 1 for k in sorted(range(18), key=lambda k: -accs[k])[:2]]  # -> [1, 13]   highest two
round(accs[12], 3)                              # -> 0.558
```




One row per configuration, sorted by the validation loss. Left: the validation loss. The configuration with the lowest validation loss is chosen. The bar on the chosen configuration spans its validation losses from the seeds 0 to 4. Right: the accuracy on the 1000 test rows, with a bar of one standard error, 0.016, on either side. A selection computes only the test accuracy of the chosen row.

The test accuracies of all 18 networks are computed here only to show how little they separate. They range from 0.534 to 0.561. An accuracy near 0.55 on 1000 rows has a standard error of $(0.55 \times 0.45 / 1000)^{0.5} = 0.016$. The whole range is less than two standard errors. Here the chosen network also has the highest test accuracy. The configuration ranked 13th by validation loss has the second highest, 0.558. Choosing by test accuracy would report the largest of 18 noisy measurements. The largest of several noisy measurements tends to be higher than what the same network reaches on new rows. The test file therefore measures only the one network chosen on the validation rows.

### FURTHER READING

- CrossEntropyLoss and NLLLoss, with the `weight` argument for class imbalance.
- torch.utils.data, for `Dataset`, `TensorDataset` and `DataLoader`.
- torch.optim, with the update rule of every optimizer written out, among them `SGD`, `Adam` and `AdamW`.
- nn.Dropout.

- Deep Learning by Goodfellow, Bengio and Courville, chapter 7 on regularisation and chapter 8 on optimization.